

Relaxed Isolation, Isolation Levels

- Transaction Management for high throughput:
- Relaxed transaction isolation
- ANSI Isolation Levels
- Phenomena

ACID Isolation

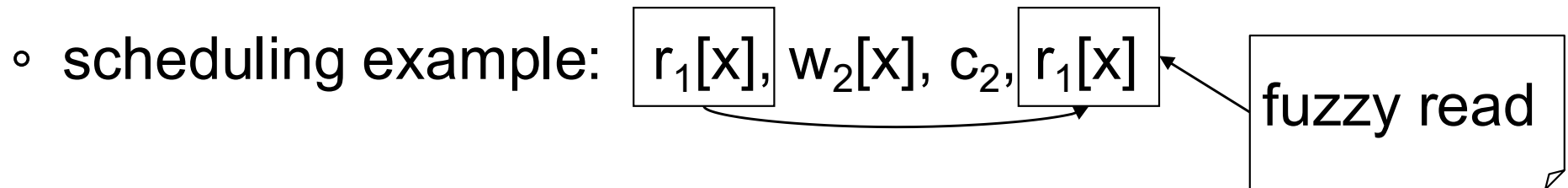
- Database operations in a transaction appear isolated from database operations of all other transactions
 - does not apply to non-database operations of client programs
- One crucial property of high isolation: **Repeatable Read**:
 - if a transaction reads a data field in the DB twice, and does not change it in between, it must receive the same value.
- Isolation with Repeatable Read is expensive:
 - leads to delays, deadlocks, and/or aborts.
 - and can be relaxed: *isolation levels*.

Repeatable Read:

- One crucial property: Repeatable Read:
 - if a transaction reads a data field in the DB twice, does not change it in between, it must receive the same value.
- Pseudocode: `val=r1[x]; wait(t); assert(val==r1[x]);`
- C++ -like language:
`val=DB.read(x); wait(t); assert(val== DB.read(x));`
- Isolation with Repeatable Read is expensive, can be relaxed.
- But always: `w1[x](val); wait(t); assert(val==r1[x]);`
`DB.write(x, val); wait(t); assert(val== DB.read(x));`
 - Writes observe the exclusive lock of other writes!

Fuzzy read

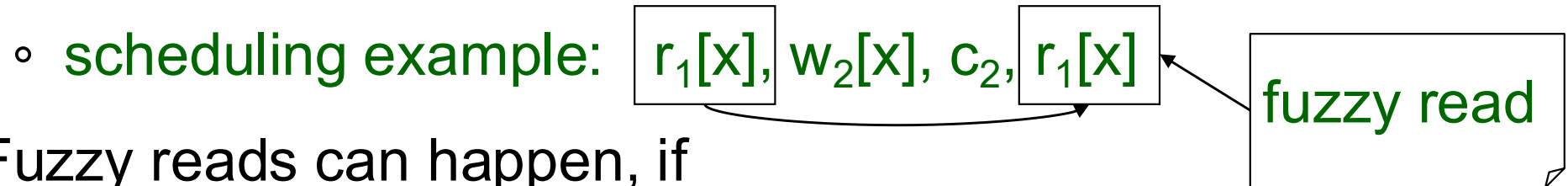
- non-repeatable read: transaction gets two different values
- `val1=DB.read(x); wait(t); val2=DB.read(x);`
- `val1` is not equal to `val2`.
- We want to call a violation of isolation that we can observe a ***phenomenon***.
- Transaction TA1 encounters a **fuzzy read** phenomenon if it reads two or more **different committed** values for x.



- Fuzzy read situations can lead to a more serious situation: lost updates.

Fuzzy read

- A type of non-repeatable read is a fuzzy read.
- Transaction TA1 encounters a fuzzy read phenomenon if it reads two or more different committed values for x .



- Fuzzy reads can happen, if
 - TA2 does not observe the read-lock of transaction TA1.
 - But TA1 observes the write lock on TA2.
- Fuzzy read situations can lead to a more serious situation (lost updates) in a rather counterfactual way:
 - If the second read does not happen or is not acted upon: coming soon

ANSI/ISO transaction isolation levels

- Isolation levels are defined with respect to three different phenomena (results of reduced isolation):
 - Dirty read: reading an uncommitted value for x.
 - Fuzzy read: reading different, committed values for x
 - Phantom: reading a new committed inserted row.
- ANSI/ISO SQL-92 defines four isolation levels:

Isolation Level	Dirty read	Fuzzy read	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Not Possible	Possible	Possible
REPEATABLE READ	Not Possible	Not Possible	Possible
SERIALIZABLE	Not Possible	Not Possible	Not Possible

Fuzzy read continued: lost updates

- a serious consequence of not using REPEATABLE READ:
a committed transaction might miss an update: **lost update**

$r_1[x]$, $r_2[x]$, $w_2[x]$, c_2 , $w_1[x]$, c_1

- example:

$r_1[x]$,

← here is the fuzzy read
“zone”

- a123 is \$99.
- TA_1 withdraws \$17, TA_2 withdraws \$23
- $r_1[x] : d_1 := 99$
- $r_2[x] : d_2 := 99$
- $w_2[x] : a123 := 76$ ($== d_2 - 23$) ← lost update
- $w_1[x] : a123 := 82$ ($== d_1 - 17$)

Preventing lost update in READ COMMITTED

Explicitly getting a write lock with SELECT FOR UPDATE can prevent lost updates even in READ COMMITTED level.

Current situation:

- s: $r_1[x], r_2[x], w_1[x], c_1, w_2[x], c_2$
- TA1: $r_1[x], w_1[x], c_1$
- TA2: $r_2[x], w_2[x]$ _____, c_2
- Second read is now SELECT FOR UPDATE
- s: $R_1[x], w_1[x], c_1, R_2[x], w_2[x], c_2$
- TA1: $R_1[x], w_1[x], c_1$
- TA2: $R_2[x]$ _____, $w_2[x], c_2$

Preventing lost update in READ COMMITTED

Explicitly getting a write lock with SELECT FOR UPDATE can prevent lost updates even in READ COMMITTED level.

Current situation:

- s: $r_1[x], r_2[x], w_1[x], c_1, w_2[x], c_2$
- TA1: $r_1[x], w_1[x], c_1$
- TA2: $r_2[x], w_2[x]$ _____, c_2
- Both reads are now SELECT FOR UPDATE
- s: $R_1[x], w_1[x], c_1, R_2[x], w_2[x], c_2$
- TA1: $R_1[x], w_1[x], c_1$
- TA2: $R_2[x]$ _____, $w_2[x], c_2$

Recap: Only one transaction uses R[]

- Can the following transactions run into a deadlock ?

- TA1: $r_1[x]$, $w_1[x]$, c_1
- TA2: $R_2[x]$, $w_2[x]$, c_2

- **No.**

Note: this was Isolation level **REPEATABLE READ**

- First s1: $r_1[x]$, $w_1[x]$, c_1 , $R_2[x]$, $w_2[x]$, c_2
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$ _____, $w_2[x]$, c_2
- 2nd s2: $R_2[x]$, $w_2[x]$, c_2 , $r_1[x]$, $w_1[x]$, c_1
 - TA1: $r_1[x]$ _____, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2

On Isolation Level READ COMMITTED

- Can the following transactions run into a deadlock ?
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2
- How do the schedules look for the two timings we have discussed before?
- First relevant timing:
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2
- Second relevant timing:
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2

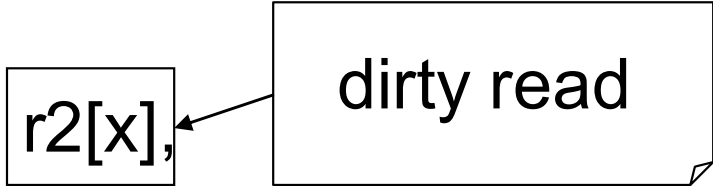
On Isolation Level READ COMMITTED

- Can the following transactions run into a deadlock ?
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2
- **Still No.** But different timing for alternative 1 on isolation level READ COMMITTED
- First s1: $r_1[x]$, $R_2[x]$, $w_2[x]$, c_2 , $w_1[x]$, c_1
 - TA1: $r_1[x]$, $w_1[x]$ _____, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2
- 2nd s2: $R_2[x]$, $w_2[x]$, c_2 , $r_1[x]$, $w_1[x]$, c_1
 - TA1: $r_1[x]$ _____, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2

On Isolation Level READ COMMITTED

- Can the following transactions run into a deadlock ?
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2
- **Still No.** But different timing for alternative 1 on isolation level READ COMMITTED
- First s1: $r_1[x]$, $R_2[x]$, **$w_2[x]$** , c_2 , $w_1[x]$, c_1 **lost update!**
 - TA1: $r_1[x]$, $w_1[x]$ _____, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2
- 2nd s2: $R_2[x]$, $w_2[x]$, c_2 , $r_1[x]$, $w_1[x]$, c_1
 - TA1: $r_1[x]$ _____, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2

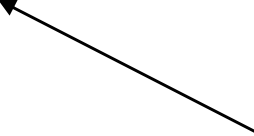
Phenomenon: Dirty read

- Transaction TA2 performs a dirty read if it reads an uncommitted write result of TA1
 - scheduling example: w1[x], r2[x],
- Dirty reads can happen, if transaction TA2 does not react to a write-lock on x.
- Dirty reads might be no problem for:
 - transactions that gather overview data
 - transactions that investigate options for later transactions
- But they are dangerous for other transactions
 - might lead to inconsistent results.
- Isolation level READ UNCOMMITTED allows dirty reads, but the transactions have to be read-only.

Dirty read on second read is not a fuzzy read

- a case, where a transaction reads two different values for x , but one of them is a dirty value:

$r_1[x], r_2[x], w_2[x], r_1[x], w_2[y], c_1$

 New, dirty value of x

- This is a dirty read and not a fuzzy read.
- The new value is not committed, so it is really no fuzzy read: Remember the definition:
- Transaction TA1 encounters a **fuzzy read** phenomenon if it reads two or more **different committed** values for x .

phantom

- A phantom (row) is a phenomenon possible in the relational data model, but goes beyond the basic read/write model.
- Are caused by relational inserts, not by updates.
- The following situation describes a phantom row:
 - TA1 performs: `SELECT * FROM mytabl`
 - gets a result set `res1`.
 - TA2 : inserts a row `r` into `mytabl` and commits.
 - TA1 performs again `SELECT * FROM mytabl`
 - gets a different result set `res2 = res1 ∪ {r}`
 - the row `r` is the phantom for TA1
- Considered less serious and hard to avoid with locks.

Summary

- We have seen the following bad phenomena of reduced isolation: phantom, fuzzy read, lost update, dirty read.
- They are increasingly serious.
- The ANSI phenomena appear in increasingly relaxed isolation levels: SERIALIZABLE, REPEATABLE READ, READ COMMITTED, READ UNCOMMITTED.
- The first level SERIALIZABLE allows no phenomenon, the last level READ UNCOMMITTED allows dirty read, fuzzy read and phantom phenomena.
- DB start on level REPEATABLE READ, on level READ COMMITTED, lost updates can be prevented with R[].